Artefato 9 – Descrição da Infraestrutura de Implantação

Sistema de apoio à gestão de requisitos com histórias de usuário

Este documento descreve o que é necessário para colocar o sistema em produção: o hardware, o software e os serviços de nuvem e de rede. O sistema é uma aplicação web em Python e Django, com dados em SQLite, em que cada projeto tem um dono e pode ter membros de outras contas.

Como pessoas de contas diferentes passam a trabalhar no mesmo projeto, o sistema precisa ficar num servidor acessível pela internet, e não só no computador de quem o executa. Por isso são descritos dois ambientes. O ambiente local serve ao desenvolvimento e à demonstração acadêmica. O ambiente de produção atende equipes pequenas, de 2 a 8 pessoas por projeto.

1 Visão geral da implantação

O navegador da pessoa usuária acessa o domínio do sistema por HTTPS. No servidor, o Nginx recebe a requisição, encerra a conexão TLS e entrega diretamente os arquivos estáticos (CSS e imagens). As demais requisições ele repassa ao Gunicorn, que executa a aplicação Django. O Django aplica autenticação, sessões, proteção CSRF e autorização por projeto, e lê e grava os dados num único arquivo SQLite no disco do servidor. Uma rotina diária copia esse arquivo para um armazenamento externo.

Componente	*Papel na implantação*
Navegador	*Exibe os templates Django em HTML/CSS e envia formulários; é o único software exigido de quem usa o sistema.*
Nginx	*Servidor web e proxy reverso: HTTPS, redirecionamento de HTTP para HTTPS, arquivos estáticos e repasse ao Gunicorn.*
Gunicorn	*Servidor de aplicação WSGI que mantém os processos da aplicação Django em execução.*
Aplicação Django	*Views, forms, regras de domínio e models do sistema, incluindo o cálculo do RICE e o controle de dono e membros.*
SQLite	*Banco de dados em arquivo (*db.sqlite3*) com as tabelas, relações e restrições definidas pelas migrations.*
Rotina de backup	*Cópia diária e consistente do banco para fora do servidor, com retenção de sete dias.*

2 Ambientes

Aspecto	Local (desenvolvimento e demonstração)	Produção
Onde roda	Computador de um integrante da equipe	Máquina virtual Linux num provedor de nuvem
Servidor web	`manage.py runserver` do Django	Nginx + Gunicorn, gerenciados pelo systemd
Banco	`db.sqlite3` na pasta do projeto	`db.sqlite3` em pasta fora do repositório, com backup diário
Acesso	`http://127.0.0.1:8000`, só nessa máquina	`https://` pelo domínio, de qualquer lugar
Configuração	`DEBUG=True`, chave de desenvolvimento	`DEBUG=False`, chave secreta em variável de ambiente, cookies seguros
Uso	Implementação, testes e vídeo de demonstração	Equipes que colaboram no mesmo projeto

3 Hardware necessário

A carga esperada é baixa: equipes pequenas, poucas requisições simultâneas e páginas sem processamento pesado. A operação mais custosa é o cálculo do RICE, uma única conta por história. Por isso, uma máquina virtual de entrada atende o sistema com folga. A meta de desempenho, de 95% de 100 operações CRUD em até 2 segundos, deve ser medida também no servidor de produção, e não só no ambiente local.

Equipamento	Configuração mínima	Justificativa
Servidor de produção	1 vCPU, 1 GB de RAM, 20 GB de disco SSD, 1 IP público	Três processos Gunicorn, o Nginx e o sistema operacional cabem em 1 GB. O banco de equipes pequenas ocupa poucos megabytes, e os 20 GB comportam sistema, aplicação, logs e sete cópias de backup.
Servidor recomendado	2 vCPU, 2 GB de RAM, 25 GB de disco SSD	Folga para picos, como várias equipes planejando sprints ao mesmo tempo, e para atualizações do sistema operacional sem degradar o atendimento.
Computador de desenvolvimento	Processador de 64 bits, 4 GB de RAM, 2 GB livres em disco	Executar Python, o servidor de desenvolvimento do Django, os testes e um editor de código.
Computador da pessoa usuária	Qualquer computador com navegador atualizado, teclado e mouse	O acesso é por navegador em computador, com telas adaptáveis a dimensões menores.

4 Software necessário

No servidor de produção

Software	Versão de referência	Uso
Ubuntu Server	24.04 LTS	Sistema operacional, com atualizações de segurança até 2029.
Python	3.12	Linguagem da aplicação, executada num ambiente virtual (venv).
Django	6.1.1	Framework web: autenticação, sessões, CSRF, ORM, migrations e templates. É a versão com que o projeto foi criado; exige Python 3.12 ou superior.
Gunicorn	Versão estável atual	Servidor de aplicação WSGI, com 3 processos (2 × vCPU + 1), executando o módulo projeto_engsoftware.wsgi:application.
Nginx	Pacote do Ubuntu	Servidor web, proxy reverso e entrega de arquivos estáticos.
SQLite	3 (embutido no Python)	Banco de dados em arquivo, com modo WAL ativado para que leituras não esperem gravações.
Certbot	Pacote do Ubuntu	Emissão e renovação automática do certificado TLS da Let's Encrypt.
systemd	Do sistema	Mantém o Gunicorn ativo, reinicia após falhas e executa o backup diário por timer.
Git e OpenSSH	Do sistema	Baixar o código do repositório no GitHub e administrar o servidor por chave SSH.
UFW	Do sistema	Firewall que libera só as portas 22, 80 e 443.

As versões exatas das bibliotecas Python ficam fixadas no requirements.txt do repositório. Assim, o ambiente local e o de produção instalam as mesmas versões.

Configuração da aplicação em produção

- DEBUG=False e SECRET_KEY lida de variável de ambiente, nunca gravada no repositório. A chave usada no desenvolvimento já foi publicada no settings.py e não serve para produção, que usa uma chave nova.
- ALLOWED_HOSTS e CSRF_TRUSTED_ORIGINS com o domínio do sistema.
- SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE e SECURE_SSL_REDIRECT ativados, para que sessão e login só trafeguem por HTTPS.

- *Banco em* /var/lib/gestao-requisitos/db.sqlite3*, fora da pasta do código, com o caminho lido de variável de ambiente em* DATABASES *e permissão de escrita só para o usuário do serviço. O arquivo do banco não é versionado no Git.*
- STATIC_ROOT *apontando para a pasta servida pelo Nginx.*
- LANGUAGE_CODE = "pt-br" *e* TIME_ZONE = "America/Sao_Paulo"*, já que a interface é em português.*
- *Painel* /admin/ *do Django restrito a quem administra o sistema, com senha forte.*
- collectstatic *para reunir os arquivos estáticos em* STATIC_ROOT *e* migrate *a cada nova versão.*

Na máquina de desenvolvimento e na da pessoa usuária

Para desenvolver são necessários Python 3.12, Git e um editor de código. As dependências são instaladas com pip install -r requirements.txt *num ambiente virtual. Quem só usa o sistema precisa apenas de um navegador atualizado, como Chrome, Firefox, Edge ou Safari, com cookies habilitados.*

5 Serviços de nuvem e de rede

Serviço	*Finalidade*	*Opções*
Máquina virtual	*Hospedar o servidor de produção descrito acima, com disco persistente.*	*Instância de entrada em AWS Lightsail, DigitalOcean, Google Cloud ou Oracle Cloud.*
IP público fixo	*Endereço estável para o domínio apontar.*	*Incluso ou contratado junto com a máquina virtual.*
Domínio e DNS	*Nome de acesso ao sistema, com registro A apontando para o IP.*	*Registro* .com.br *no Registro.br ou subdomínio gratuito do provedor.*
Certificado TLS	*HTTPS, exigido para proteger senhas e cookies de sessão.*	*Let's Encrypt, gratuito, via Certbot.*
Armazenamento de backup	*Guardar fora do servidor as cópias diárias do banco.*	*Armazenamento de objetos do mesmo provedor ou outro disco remoto.*
Repositório Git	*Versionar o código e servir de origem para cada implantação.*	*GitHub (repositório EngSoftware-26-2/Project_EngSoftware).*
Firewall de rede	*Liberar apenas SSH (22, só com chave), HTTP (80, redirecionado) e HTTPS (443).*	*UFW no servidor e regras de rede do provedor.*

O sistema não envia e-mails nem se integra a serviços externos. Notificações e integrações estão fora do escopo, e o convite de membros é feito pelo e-mail de uma conta já existente, sem envio de mensagem. Por isso, nenhum serviço de e-mail é necessário.

Para a entrega acadêmica, uma alternativa mais simples à máquina virtual é uma plataforma de hospedagem Python com disco persistente, como o PythonAnywhere. Ela já fornece servidor web, HTTPS e subdomínio. A plataforma escolhida precisa manter o arquivo SQLite entre reinícios. Serviços com sistema de arquivos temporário apagariam o banco a cada nova versão.

6 Procedimento de implantação

1. *Criar a máquina virtual com Ubuntu Server 24.04, cadastrar a chave SSH, ativar o UFW e instalar Python, Nginx, Certbot e Git.*
2. *Clonar o repositório, criar o ambiente virtual e instalar o Gunicorn e as dependências listadas no* requirements.txt.
3. *Definir as variáveis de ambiente de produção e criar a pasta do banco fora do código.*
4. *Executar* migrate *e* collectstatic.
5. *Criar o serviço do Gunicorn no systemd e o site no Nginx, apontando para o socket do Gunicorn e para a pasta de estáticos.*
6. *Apontar o registro A do domínio para o IP e emitir o certificado com o Certbot.*
7. *Ativar o timer de backup diário e testar a restauração de uma cópia.*
8. *Verificar cadastro, login, criação de projeto, adição de membro e cálculo do RICE pelo domínio, com HTTPS.*

Cada nova versão segue os passos 2 e 4: atualizar o código pelo Git, instalar dependências, aplicar migrations, recolher estáticos e reiniciar o serviço do Gunicorn.

7 Limites e evolução

O SQLite atende o volume previsto porque grava uma transação por vez e as gravações deste sistema são curtas. O limite aparece com muitas equipes gravando ao mesmo tempo. Nesse caso, a evolução natural é trocar o banco por PostgreSQL, o que no Django muda a configuração DATABASES *e não exige reescrever models nem views.*

Um único servidor é ponto único de falha. O backup diário com teste de restauração limita a perda a, no máximo, um dia de dados. Alta disponibilidade, balanceamento de carga e escalonamento automático ficam fora do escopo.